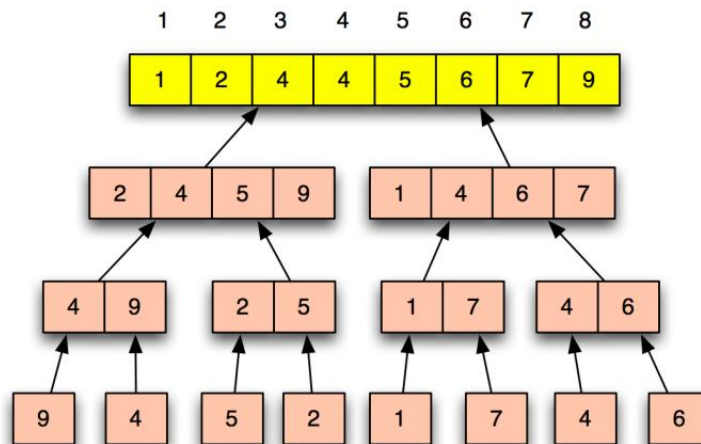
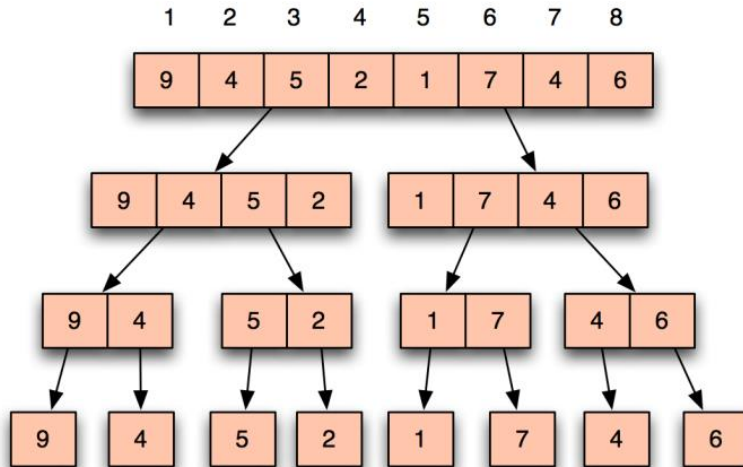


Merge Sort



$$T(n) = 2T(n/2) + \Theta(n)$$

subproblems subproblem size work dividing and combining

$$O(n \log n)$$

In **Merge Sort**, the given unsorted array with **n** elements, is divided into **n** subarrays, each having **one** element, because a single element is always sorted in itself. Then, it repeatedly merges these subarrays, to produce new sorted subarrays, and in the end, one complete sorted array is produced.

The concept of Divide and Conquer involves three steps:

1. **Divide** the problem into multiple small problems.
2. **Conquer** the subproblems by solving them. The idea is to break down the problem into atomic subproblems, where they are actually solved.
3. **Combine** the solutions of the subproblems to find the solution of the actual problem.

How Merge Sort Works?

As we have already discussed that merge sort utilizes divide-and-conquer rule to break the problem into sub-problems, the problem in this case being, **sorting a given array**.

In merge sort, we break the given array midway, for example if the original array had **6** elements, then merge sort will break it down into two subarrays with **3** elements each.

But breaking the original array into 2 smaller subarrays is not helping us in sorting the array.

So we will break these subarrays into even smaller subarrays, until we have multiple subarrays with **single element** in them. Now, the idea here is that an array with a single element is already sorted, so once we break the original array into subarrays which has only a single element, we have successfully broken down our problem into base problems.

And then we have to merge all these sorted subarrays, step by step to form one single sorted array.

Let's consider an array with values **{14, 7, 3, 12, 9, 11, 6, 12}**

Below, we have a pictorial representation of how merge sort will sort the given array.

In merge sort we follow the following steps:

1. We take a variable `p` and store the starting index of our array in this. And we take another variable `r` and store the last index of array in it.
2. Then we find the middle of the array using the formula $(p + r)/2$ and mark the middle index as `q`, and break the array into two subarrays, from `p` to `q` and from `q + 1` to `r` index.
3. Then we divide these 2 subarrays again, just like we divided our main array and this continues.
4. Once we have divided the main array into subarrays with single elements, then we start merging the subarrays.

```
def mergesort(arr):
    if len(arr) <= 1:
        return arr
    else:
        mid = len(arr) // 2
        a1 = mergesort(arr[:mid])
        a2 = mergesort(arr[mid:])
        return merge(a1, a2)

def merge(a1, a2):
    merged = [0] * (len(a1) + len(a2))
    i = j = k = 0
    while i < len(a1) and j < len(a2):
        if a1[i] < a2[j]:
            merged[k] = a1[i]
            i += 1
        else:
            merged[k] = a2[j]
            j += 1
        k += 1
    while i < len(a1):
        merged[k] = a1[i]
        i += 1
        k += 1
    while j < len(a2):
        merged[k] = a2[j]
        j += 1
        k += 1
    return merged
```

Merge Sort Complexity

Time Complexity

Best $O(n \cdot \log n)$

Worst $O(n \cdot \log n)$

Average $O(n \cdot \log n)$

Space Complexity $O(n)$

Stability Yes